UNIVERSITY
OF COLOGNE

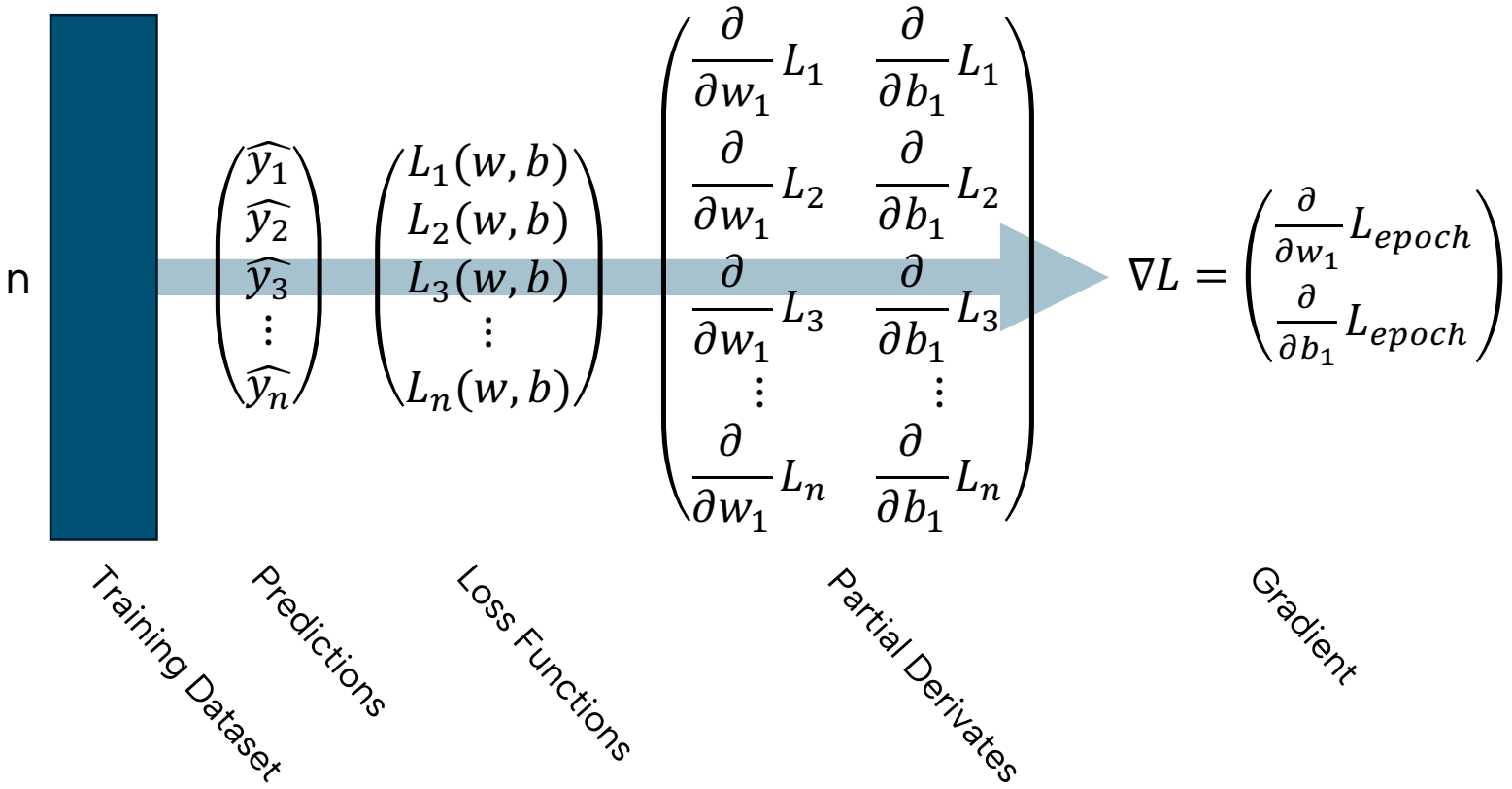
WORKSHOP 06

Advanced Analytics and Applications

Agenda

1	**Lecture Recap**
2	Exemplary Exam Questions
3	Coding Tasks

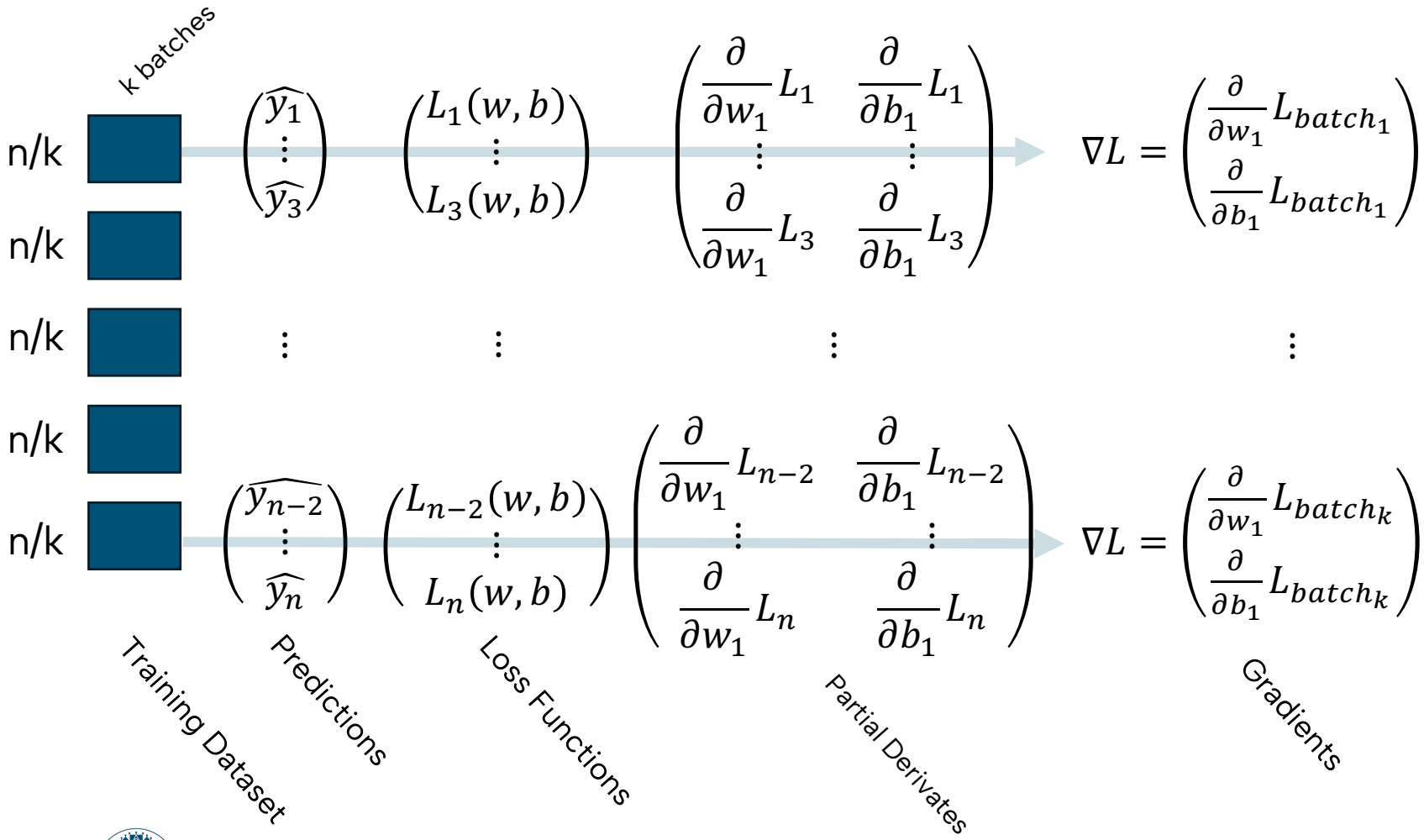
Gradient Descent



If we would do **gradient descent** on the full training data set (epoch), every full gradient step would be pretty "robust" but also rather tiny.

The more problematic thing is usually computation complexity – it's simply too hard to compute millions of partial derivatives at the same time and keep them in memory!

Stochastic Gradient Descent (SGD)



In **stochastic gradient descent** we do updates in batches of the training set, the epoch consists of multiple, incremental updates.

Momentum

- Because updates in SGD are incremental and an approximation of the true gradient they are more "chaotic". If we simply use learning rate η as before, convergence takes a long time, and there is a risk of overstepping:

$$\theta_{i+1} = \theta_i - \eta \nabla L(\theta_i)$$

- Instead, we can use momentum, which considers past updates as indicative of a "right direction". For this, we introduce a parameter α, which decays past experiences (how important they are). The current velocity / momentum is then

$$v_i = \alpha v_{i-1} + \eta \nabla L(\theta_i)$$

- The update rule becomes

$$\theta_{i+1} = \theta_i - v_i$$

- Importantly, momentum is **per-parameter**, which makes it "explore" different coefficients differently! (with all the up- and downsides of that, which is for you to think about)

Agenda

1	Lecture Recap
2	**Exemplary Exam Questions**
3	Coding Tasks

Multiple Choice Questions: Learning

Question: Which of the following statements correctly describes how optimization techniques like Stochastic Gradient Descent (SGD), learning rate, or momentum function?

- ❑ Setting the learning rate very high ensures the algorithm will converge to the optimum faster.
- ❑ SGD calculates the gradient by using the entire training dataset during each iteration.
- ❑ Momentum accelerates learning by remembering the last update and combining it with the current gradient.
- ❑ Gradient descent algorithms are mathematically guaranteed to find the global optimum regardless of the initial starting point.

Multiple Choice Questions: Learning

Question: Which of the following statements correctly describes how optimization techniques like Stochastic Gradient Descent (SGD), learning rate, or momentum function?

- ☐ Setting the learning rate very high ensures the algorithm will converge to the optimum faster.
- ☐ SGD calculates the gradient by using the entire training dataset during each iteration.
- ☑ Momentum accelerates learning by remembering the last update and combining it with the current gradient.
- ☐ Gradient descent algorithms are mathematically guaranteed to find the global optimum regardless of the initial starting point.

Question 1: Gradient Descent

1. $\nabla(x^2 + y^2 + x) = \begin{pmatrix} 2x + 1 \\ 2y \end{pmatrix}$ Partial Derivative

2. $\nabla(x^2 + y^2 + x) = \begin{pmatrix} 2x + 1 \\ 2y \end{pmatrix} =^! \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ First Order Cond.

3. $\begin{pmatrix} x* \\ y* \end{pmatrix} = \begin{pmatrix} -1/2 \\ 0 \end{pmatrix}$ Solve for extrema

Task 1): Analytically calculate the minima of the following function. You don't need to verify whether the critical point is a minima using the hessian matrix.

$$x^2 + y^2 + x$$

Question 1: Gradient Descent

1. $$\begin{pmatrix} x_0 \\ y_0 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

2. $$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \end{pmatrix} - 0.3 * \begin{pmatrix} 2 * (-1) + 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.7 \\ 0 \end{pmatrix}$$

3. $$\begin{pmatrix} x_2 \\ y_2 \end{pmatrix} = \begin{pmatrix} -0.7 \\ 0 \end{pmatrix} - 0.3 * \begin{pmatrix} 2 * (-0.7) + 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.58 \\ 0 \end{pmatrix}$$

Task 2): Now, for the same function, calculate the minima using the gradient descent algorithm. Select a learning rate of 0.3, (x0, y0) = (–1, 0) as the starting point, and terminate after 2 iterations.

$$x^2 + y^2 + x$$

$$\nabla(x^2 + y^2 + x) = \begin{pmatrix} 2x + 1 \\ 2y \end{pmatrix}$$

Question 1: Gradient Descent

1. $$\begin{pmatrix} x_0 \\ y_0 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

2. $$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \end{pmatrix} - 1 * \begin{pmatrix} 2 * (-1) + 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

3. $$\begin{pmatrix} x_2 \\ y_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 1 * \begin{pmatrix} 2 * (0) + 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

What happens now?
We oscillate back and forth between the two solutions!

Task 3): What happens when we set the learning rate to 1?

$$x^2 + y^2 + x$$

$$\nabla(x^2 + y^2 + x) = \begin{pmatrix} 2x + 1 \\ 2y \end{pmatrix}$$

Question 2: Augmentation in NNs

Question: Explain in your own words, why *parameter sharing* in neural networks can be beneficial.

Answer: When parameters are shared, the model is encouraged to learn **features that are useful across different parts** of the input or even across different tasks. This means less parameters and thus **less memory**! Essentially, this has a **regularization effect** on the network!

Question 3: Optimizers

Question: Momentum keeps track of every coefficient θ (weights and biases) with an individual v. Still, it uses the same parameters (learning rate η and decay α) for all of these.

a) What problem do you identify here?

b) Can you think of a solution?

Answer:

a) We might run into a problem of **heterogeneous feature frequency**: Some weights/biases (which represent learned features) could be important (e.g. the color of a pixel –> every training observation has that) and thus have rather large updates from the gradient in each iteration while others are rarely impacted. The problem is the following: With a small decay/large learning rate these updates will be massive, they might overstep and/or converge (too) fast. Intuition would be to lower η or increase α. This would however negatively impact the ability to learn infrequent features.
One choice of hyperparameters might be correct for certain features but not for others!

b) We could track the total accumulated sum of gradient for all θ instead of computing v and use that to proportionally increase/lower the feature-individual learning rate η_θ **–> Essentially this is AdaGrad**

AdaGrad

Paper: Duchi, J., Hazan, E., Singer, Y., 2011. Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. J. Mach. Learn. Res. 12, 2121–2159. https://dl.acm.org/doi/10.5555/1953048.2021068

- Let's have a look at that "solution" to the problem of feature frequency in-depth.
- AdaGrad stands for Adaptive Gradient and it uses the gradient history to adaptively change the learning rate for every parameter!
- For every parameter θ we track the squared sum of past gradients:

$$g_i = g_{i-1} + \nabla L(\theta_i)^2$$

- The update rule becomes

$$\theta_{i+1} = \theta_i - \frac{\eta}{\sqrt{g_i} + \varepsilon} \nabla L(\theta_i)$$

where ε is a small term that prevents division by zero… and that already brings us to the

Question: Can you identify a problem with AdaGrad?

Answer: Because we sum up gradients over time, the learning rate will effectively shrink to ε / 0, stopping learning altogether.

RMSProp

Paper: Hinton, G., n.d. Neural Networks for Machine Learning. Lecture Notes.
https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

- To fix that, RMSprop (Root Mean Squared Propagation) adds the idea of decay to that of adaptive learning rates based on gradients.
- The gradient cache for coefficient θ becomes
$$g_i = \alpha g_{i-1} + (1-\alpha)\nabla L(\theta_i)^2$$
where α is a decay factor like 0.99.
- The update rule stays
$$\theta_{i+1} = \theta_i - \frac{\eta}{\sqrt{g_i} + \varepsilon}\nabla L(\theta_i)$$
- **Question:** So… is RMSprop the perfect optimizer?
- **Answer:** No, it misses the notion of momentum. It can still sharply sidestep the direction of the general gradient direction if one (mini-)batch has a very different gradient.
(And it can get stuck in local minima / plateaus of the loss function – when the gradient is close to zero the learning rate is scaled up but it's still multiplied with the close-to-zero gradient).

Adam

Paper: Kingma, D.P., Ba, J., 2017. Adam: A Method for Stochastic Optimization. https://doi.org/10.48550/arXiv.1412.6980

- …which brings us to combine the two notions: Track both momentum and adaptive learning rates!
- The **adam** optimizer does that by tracking

$$v_i = \alpha_v v_{i-1} + (1 - \alpha_v)\nabla L(\theta_i)$$
$$g_i = \alpha_g g_{i-1} + \left(1 - \alpha_g\right)\nabla L(\theta_i)^2$$

- Additionally, we apply a correction because v_i and g_i are typically initialized as zero vectors meaning they are biased towards no (or small) updates in the first iterations:

$$\widehat{v_i} = \frac{v_i}{1 - \alpha_v^i}$$
$$\widehat{g_i} = \frac{g_i}{1 - \alpha_g^i}$$

- The update rule becomes:

$$\theta_{i+1} = \theta_i - \frac{\eta}{\sqrt{g_i} + \varepsilon}\widehat{v_i}$$

Agenda

1	Lecture Recap
2	Exemplary Exam Questions
3	**Coding Tasks**

Contact

For general questions and enquiries on **research**, **teaching**, **job openings** and **new projects** refer to our website at www.is3.uni-koeln.de

For specific **enquiries regarding this course** contact us by sending an email to the **IS3 teaching address** at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>